

Laboratorio 7

Integrantes

Nombre	Carnet	Usuario Git
Edwin Jose Gabriel De Leon Garcia	22809	EJGDLG
Gustavo Adolfo Cruz Bardales	22779	G2309
Josué Emanuel Say Garcia	22801	JosueSay
Mathew Alexander Cordero Aquino	22982	donmatthiuz

Repositorio

[Link al Repositorio](#)

Caso

Una empresa de ciberseguridad opera un sistema de detección de intrusiones en redes corporativas. El sistema monitorea el tráfico de red en tiempo real y debe decidir qué acción tomar ante comportamientos sospechosos: ignorar, alertar, bloquear temporalmente, o aislar el segmento afectado. El estado del sistema en cada instante es un vector de alta dimensión que incluye métricas de tráfico, patrones de comportamiento de usuarios, y logs de eventos recientes. La empresa quiere explorar si Deep RL puede mejorar la toma de decisiones frente a los sistemas basados en reglas que usan actualmente.

Task 1

Task 1.1

El equipo de ingeniería propone usar DQN estándar para este sistema. Antes de implementar cualquier cosa, usted debe hacer un análisis crítico de esa propuesta.

a. El estado del sistema es un vector de 256 dimensiones con valores continuos. Argumenten si DQN es arquitecturalmente apropiado para este tipo de estado. ¿Qué tipo de red usarían como aproximador de $Q(s, a; w)$ y por qué, considerando que el estado no es una imagen sino un vector numérico?

Respuesta: DQN es arquitecturalmente apropiado porque reemplaza la tabla Q de Q-learning por una red neuronal que aproxima $Q(s, a; w)$. Una tabla Q no es viable: un estado con 256 variables continuas produce un espacio de estados demasiado grande e imposible de enumerar. Se usaría una red feedforward o MLP totalmente conectada, con 256 entradas y cuatro salidas, una por acción. Esta arquitectura permite aprender relaciones entre las métricas, patrones de usuarios y logs del vector. No se requiere una CNN porque el estado no tiene estructura espacial como una imagen. Tampoco se requiere una red recurrente si el vector incorpora suficiente historial reciente para representar aproximadamente el estado de Markov.

b. El espacio de acciones tiene 4 opciones discretas. Argumenten si DQN o alguna de sus variantes es apropiado para este espacio de acciones, o si es necesario un algoritmo diferente.

Respuesta: El espacio de acciones es discreto y finito: ignorar, alertar, bloquear temporalmente y aislar el segmento. Por ello DQN es apropiado, ya que puede producir un valor Q por cada acción y el máximo del target se calcula sobre solo cuatro opciones. No es necesario usar algoritmos para control continuo, como DDPG o SAC. Sin embargo, Double DQN sería una mejora recomendable frente a DQN estándar, porque reduce la sobreestimación al separar la selección de la mejor acción con la red principal y su evaluación con la red objetivo.

c. En el dominio de detección de intrusiones, los eventos críticos (intrusiones reales) son extremadamente raros: ocurren en menos del 0.1% de los pasos de tiempo. Argumenten formalmente qué consecuencia tiene esa rareza sobre el buffer de Experience Replay y sobre la distribución de muestreo uniforme. ¿Qué variante de DQN resolvería este problema específicamente y cómo?

Respuesta: Sea $p = 0.001$ la proporción máxima de intrusiones reales en el buffer. Bajo muestreo uniforme, una transición crítica tiene probabilidad p de ser seleccionada y un minibatch de tamaño B contiene en promedio Bp intrusiones. Por ejemplo, para $B = 32$, el valor esperado es $32(0.001) = 0.032$ intrusiones por minibatch y la probabilidad de no incluir ninguna es aproximadamente $(1 - 0.001)^{32} \approx 96.8\%$. Por tanto, el Experience Replay queda dominado por tráfico normal y el agente recibe muy poca señal de aprendizaje sobre eventos críticos. Prioritized Experience Replay mitiga este problema al muestrear con mayor probabilidad las transiciones de mayor error TD:

$P(i) \propto (|\delta_i| + \epsilon)^\alpha$. Las intrusiones, especialmente al inicio, suelen generar errores TD altos por ser inesperadas, por lo que se reutilizan más frecuentemente que bajo muestreo uniforme.

d. El sistema actual basado en reglas tiene una tasa de falsos positivos del 8%. Si diseñaran la función de recompensa de DQN para minimizar falsos positivos únicamente, ¿qué comportamiento indeseable podría aprender el agente? Diseñen una función de recompensa con al menos tres componentes que capturen el objetivo real del sistema, justificando la magnitud relativa de cada uno.

Respuesta:

Si la recompensa minimizara únicamente los falsos positivos, el agente podría aprender a ignorar la mayoría de los comportamientos sospechosos. Esto reduciría alertas incorrectas, pero incrementaría los falsos negativos al dejar intrusiones reales sin detectar ni contener. Una función de recompensa debe balancear detección, contención y continuidad operativa:

$$r = 40I_{\text{intrusión detectada}} + 80I_{\text{intrusión contenida}} - 200I_{\text{intrusión ignorada}} - 20I_{\text{acción defensiva sobre tráfico legítimo}} - c(a)$$

$$c(\text{ignorar}) = 0, \quad c(\text{alertar}) = 1, \quad c(\text{bloquear}) = 5, \quad c(\text{aislar}) = 15$$

La penalización por ignorar una intrusión es la mayor porque un falso negativo puede causar daño grave. Detectarla tiene una recompensa positiva, y contenerla tiene una recompensa adicional para que el agente no se limite a alertar cuando bloquear o aislar sea necesario. Los falsos positivos se penalizan para mejorar la tasa actual del 8%, mientras que el costo creciente de las acciones evita bloqueos o aislamientos innecesarios.

Task 1.2

El equipo propone usar Double DQN en lugar de DQN estándar.

a. Expliquen formalmente el problema de sobreestimación que Double DQN resuelve. En el contexto específico de detección de intrusiones, ¿qué consecuencia operacional tendría que el agente sobreestime sistemáticamente el valor de la acción "bloquear"? ¿Y de la acción "ignorar"?

Respuesta:

DQN estándar puede sobreestimar los valores Q porque utiliza la misma red objetivo w^- para seleccionar la acción con mayor valor y para evaluar ese valor en el target. Si las estimaciones contienen ruido, el operador $\max$ tiende a escoger acciones cuyo error es positivo, produciendo una sobreestimación sistemática. Double DQN reduce este sesgo al separar la selección y la evaluación de la acción. En detección de intrusiones, sobreestimar el valor de bloquear puede llevar al agente a bloquear tráfico legítimo más de lo necesario, causando interrupciones operativas y falsos positivos. Sobreestimar el valor de ignorar puede hacer que el agente considere inofensivos eventos que requieren respuesta, aumentando falsos negativos y permitiendo que intrusiones reales continúen sin contención.

b. Escriban la expresión del target de DQN estándar y la del target de Double DQN, identificando con precisión qué parámetros hacen qué en cada caso. ¿Cuál es el único cambio de implementación entre ambos?

Respuesta:

El target de DQN estándar es:

$$y_{\text{DQN}} = r + \gamma \max_{a'} Q(s', a'; w^-)$$

Aquí, la red objetivo con parámetros w^- selecciona la acción con mayor valor y también evalúa esa misma acción.

El target de Double DQN es:

$$y_{\text{Double DQN}} = r + \gamma Q\left(s', \arg \max_{a'} Q(s', a'; w); w^-\right)$$

En Double DQN, la red principal con parámetros w selecciona la acción mediante $\arg \max$, mientras que la red objetivo con parámetros w^- evalúa el valor de la acción seleccionada. El único cambio de implementación consiste en usar la red principal para seleccionar la acción del target; la red objetivo sigue congelada y se actualiza periódicamente como en DQN estándar.

c. Argumenten si en este dominio específico la sobreestimación es un problema más grave en ciertos tipos de estados que en otros. Consideren estados con alta ambigüedad (tráfico borderline) versus estados claramente maliciosos.

Respuesta:

La sobreestimación es más grave en estados ambiguos, como tráfico borderline, porque las señales disponibles no distinguen claramente entre actividad legítima y maliciosa. En esos estados, las estimaciones Q tienen mayor incertidumbre y ruido; al aplicar el operador $\max$, DQN tiene mayor probabilidad de elegir una acción cuyo valor está inflado por error. Por ejemplo, podría sobreestimar bloquear aunque la evidencia sea insuficiente. En estados claramente maliciosos, las diferencias entre las acciones correctas e incorrectas deberían ser más marcadas, por lo que el ruido relativo y el riesgo de seleccionar un valor inflado son menores. Por ello, Double DQN es especialmente útil para evitar decisiones excesivas ante eventos ambiguos.

Task 2

Task 2.1

Implementen DQN y Double DQN sobre el entorno LunarLander-v2 de Gymnasium, que tiene espacio de acción discreto con 4 acciones y estado continuo de 8 dimensiones. Usen este entorno como proxy del sistema de detección de intrusiones para validar las propiedades algorítmicas antes de escalar al dominio real.

La red para $Q(s, a; w)$ debe tener dos capas ocultas de 128 neuronas con activación ReLU. El buffer de Experience Replay debe tener capacidad de 50,000 transiciones. El minibatch debe ser de 64 transiciones. La Target Network debe actualizarse cada 500 pasos. Usen ϵ decreciente de 1.0 a 0.05 durante los primeros 10,000 pasos.

Ambos algoritmos deben implementarse desde cero usando PyTorch. No se permite usar implementaciones preconstruidas de DQN de ninguna librería de RL.

La implementación debe registrar por episodio: recompensa total, valor Q promedio estimado sobre un conjunto fijo de 100 estados de evaluación, y el target promedio usado en las actualizaciones. Estos tres registros son necesarios para detectar sobreestimación.

Task 2.1

LunarLander-v2 ya no existe en la versión actual de Gymnasium. Se usa LunarLander-v3, que es el mismo entorno con 8 dimensiones de estado y 4 acciones.

Los dos algoritmos comparten todo el código. La única diferencia está en cómo se calcula el target: DQN elige y evalúa la acción siguiente con la red target, Double DQN la elige con la red online y la evalúa con la red target.

Los tres registros por episodio se calculan así:

Registro	Cálculo
Recompensa total	Suma de recompensas del episodio
Q promedio	Promedio de $\max_a Q(s, a; w)$ con la red online sobre 100 estados fijos, medido al cerrar cada episodio
Target promedio	Promedio de y sobre todos los minibatches usados para actualizar durante el episodio

Los 100 estados de evaluación se recolectan una sola vez con una política aleatoria y semilla fija, así ambos algoritmos se miden sobre exactamente los mismos estados.

Hiperparámetros que la tarea no fija: $\gamma = 0.99$, tasa de aprendizaje de 5×10^{-4} con Adam, pérdida Huber, recorte de gradiente en norma 10 y 1,000 transiciones en el buffer antes de empezar a actualizar. Cada algoritmo entrena 600 episodios con la semilla 42.

```
import random

import numpy as np
import torch
import torch.nn as nn
import gymnasium as gym
import pandas as pd

pd.set_option("display.width", 200)

ENV_ID = "LunarLander-v3"
SEED = 42
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

HIDDEN_SIZE = 128
BUFFER_CAPACITY = 50_000
BATCH_SIZE = 64
TARGET_UPDATE_STEPS = 500
EPS_START, EPS_END, EPS_DECAY_STEPS = 1.0, 0.05, 10_000
GAMMA = 0.99
LR = 5e-4
LEARNING_STARTS = 1_000
NUM_EPISODES = 600
NUM_EVAL_STATES = 100

def set_global_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
```

```

class QNetwork(nn.Module):
    def __init__(self, obs_dim, n_actions):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(obs_dim, HIDDEN_SIZE),
            nn.ReLU(),
            nn.Linear(HIDDEN_SIZE, HIDDEN_SIZE),
            nn.ReLU(),
            nn.Linear(HIDDEN_SIZE, n_actions),
        )

    def forward(self, state):
        return self.net(state)

class ReplayBuffer:
    def __init__(self, capacity, obs_dim):
        self.capacity = capacity
        self.states = np.zeros((capacity, obs_dim), dtype=np.float32)
        self.actions = np.zeros(capacity, dtype=np.int64)
        self.rewards = np.zeros(capacity, dtype=np.float32)
        self.next_states = np.zeros((capacity, obs_dim), dtype=np.float32)
        self.dones = np.zeros(capacity, dtype=np.float32)
        self.pos = 0
        self.size = 0

    def add(self, state, action, reward, next_state, done):
        self.states[self.pos] = state
        self.actions[self.pos] = action
        self.rewards[self.pos] = reward
        self.next_states[self.pos] = next_state
        self.dones[self.pos] = done
        # buffer circular: al llenarse se sobrescribe la transición más vieja
        self.pos = (self.pos + 1) % self.capacity
        self.size = min(self.size + 1, self.capacity)

    def sample(self, batch_size):
        idx = np.random.randint(0, self.size, size=batch_size)
        return (
            torch.as_tensor(self.states[idx], device=DEVICE),
            torch.as_tensor(self.actions[idx], device=DEVICE),
            torch.as_tensor(self.rewards[idx], device=DEVICE),
            torch.as_tensor(self.next_states[idx], device=DEVICE),
            torch.as_tensor(self.dones[idx], device=DEVICE),
        )

```

```

def epsilon_at(step):
    frac = min(step / EPS_DECAY_STEPS, 1.0)
    return EPS_START + frac * (EPS_END - EPS_START)

def collect_eval_states(n_states, seed):
    env = gym.make(ENV_ID)
    rng = np.random.default_rng(seed)
    states = []
    obs, _ = env.reset(seed=seed)
    while len(states) < 10 * n_states:
        states.append(obs)
        obs, _, terminated, truncated, _ = env.step(int(rng.integers(env.action_space.n)))
        if terminated or truncated:
            obs, _ = env.reset()
    env.close()
    idx = rng.choice(len(states), size=n_states, replace=False)
    return torch.as_tensor(np.array(states)[idx], dtype=torch.float32, device=DEVICE)

def compute_targets(online_net, target_net, rewards, next_states, dones, double):
    with torch.no_grad():
        if double:
            # Double DQN: La red online elige a*, la red target la evalúa
            best_actions = online_net(next_states).argmax(dim=1, keepdim=True)
            next_q = target_net(next_states).gather(1, best_actions).squeeze(1)
        else:
            # DQN: La red target elige y evalúa con el mismo max
            next_q = target_net(next_states).max(dim=1).values
    return rewards + GAMMA * (1.0 - dones) * next_q

```

```

def train_dqn(double, eval_states, num_episodes=NUM_EPISODES, seed=SEED):
    set_global_seed(seed)
    env = gym.make(ENV_ID)
    env.action_space.seed(seed)
    obs_dim = env.observation_space.shape[0]
    n_actions = env.action_space.n

    online_net = QNetwork(obs_dim, n_actions).to(DEVICE)
    target_net = QNetwork(obs_dim, n_actions).to(DEVICE)
    target_net.load_state_dict(online_net.state_dict())
    optimizer = torch.optim.Adam(online_net.parameters(), lr=LR)
    buffer = ReplayBuffer(BUFFER_CAPACITY, obs_dim)

    history = {"reward": [], "q_eval": [], "target": []}
    global_step = 0
    obs, _ = env.reset(seed=seed)
    name = "Double DQN" if double else "DQN"

    for episode in range(num_episodes):
        if episode > 0:
            obs, _ = env.reset()
            episode_reward = 0.0
            episode_targets = []
            done = False

            while not done:
                if random.random() < epsilon_at(global_step):
                    action = env.action_space.sample()
                else:
                    with torch.no_grad():
                        state_t = torch.as_tensor(obs, dtype=torch.float32, device=DEVICE)
                        action = int(online_net(state_t).argmax().item())

                next_obs, reward, terminated, truncated, _ = env.step(action)
                # solo terminated corta el bootstrap, truncated es un corte por límite de tiempo
                buffer.add(obs, action, reward, next_obs, float(terminated))
                obs = next_obs
                episode_reward += reward
                done = terminated or truncated
                global_step += 1

            if buffer.size >= LEARNING_STARTS:
                states, actions, rewards, next_states, dones = buffer.sample(BATCH_SIZE)
                targets = compute_targets(online_net, target_net, rewards, next_states, dones, double)
                q_taken = online_net(states).gather(1, actions.unsqueeze(1)).squeeze(1)
                loss = nn.functional.smooth_l1_loss(q_taken, targets)
                optimizer.zero_grad()
                loss.backward()
                nn.utils.clip_grad_norm_(online_net.parameters(), 10.0)
                optimizer.step()
                episode_targets.append(targets.mean().item())

            if global_step % TARGET_UPDATE_STEPS == 0:
                target_net.load_state_dict(online_net.state_dict())

        with torch.no_grad():
            q_eval = online_net(eval_states).max(dim=1).values.mean().item()

        history["reward"].append(episode_reward)
        history["q_eval"].append(q_eval)
        history["target"].append(float(np.mean(episode_targets)) if episode_targets else np.nan)

    if (episode + 1) % 50 == 0:
        print(

```

```

        f"{name} episodio {episode + 1}/{num_episodes} pasos {global_step} "
        f"recompensa (50) {np.mean(history['reward'][-50:]):7.1f} "
        f"Q {q_eval:6.2f} target {history['target'][-1]:6.2f}"
    )

    env.close()
    return online_net, {k: np.array(v) for k, v in history.items()}

```

```
eval_states = collect_eval_states(NUM_EVAL_STATES, SEED)
```

```
dqn_net, dqn_history = train_dqn(double=False, eval_states=eval_states)
ddqn_net, ddqn_history = train_dqn(double=True, eval_states=eval_states)
```

```

DQN episodio 50/600 pasos 8088 recompensa (50) -129.2 Q 6.33 target 5.76
DQN episodio 100/600 pasos 56239 recompensa (50) 8.0 Q -30.90 target 12.69
DQN episodio 150/600 pasos 88992 recompensa (50) 154.4 Q 35.22 target 12.54
DQN episodio 200/600 pasos 119540 recompensa (50) 167.7 Q 37.78 target 13.00
DQN episodio 250/600 pasos 148639 recompensa (50) 218.7 Q 36.81 target 13.26
DQN episodio 300/600 pasos 174972 recompensa (50) 230.4 Q 40.41 target 16.88
DQN episodio 350/600 pasos 196831 recompensa (50) 240.2 Q 61.95 target 19.35
DQN episodio 400/600 pasos 221001 recompensa (50) 216.3 Q 70.10 target 19.81
DQN episodio 450/600 pasos 242886 recompensa (50) 247.5 Q 67.12 target 21.30
DQN episodio 500/600 pasos 259560 recompensa (50) 251.1 Q 71.38 target 24.74
DQN episodio 550/600 pasos 273035 recompensa (50) 216.9 Q 79.45 target 31.10
DQN episodio 600/600 pasos 288311 recompensa (50) 250.7 Q 79.90 target 36.02
Double DQN episodio 50/600 pasos 6170 recompensa (50) -125.8 Q 3.30 target 6.72
Double DQN episodio 100/600 pasos 55875 recompensa (50) -34.6 Q -12.82 target 11.96
Double DQN episodio 150/600 pasos 104598 recompensa (50) -10.3 Q 62.27 target 3.95
Double DQN episodio 200/600 pasos 131849 recompensa (50) 165.8 Q 85.64 target 24.13
Double DQN episodio 250/600 pasos 155011 recompensa (50) 175.9 Q 70.69 target 31.03
Double DQN episodio 300/600 pasos 177887 recompensa (50) 199.7 Q 65.97 target 27.95
Double DQN episodio 350/600 pasos 197295 recompensa (50) 208.2 Q 73.19 target 24.87
Double DQN episodio 400/600 pasos 219745 recompensa (50) 211.1 Q 85.23 target 24.80
Double DQN episodio 450/600 pasos 242968 recompensa (50) 210.2 Q 61.22 target 22.63
Double DQN episodio 500/600 pasos 258609 recompensa (50) 256.9 Q 41.00 target 26.81
Double DQN episodio 550/600 pasos 270082 recompensa (50) 232.4 Q 54.22 target 30.58
Double DQN episodio 600/600 pasos 278747 recompensa (50) 110.9 Q 75.25 target 37.81

```

```

def summarize(history, block=100):
    rows = []
    for start in range(0, len(history["reward"]), block):
        end = start + block
        rows.append({
            "episodios": f"{start + 1}-{end}",
            "recompensa": np.mean(history["reward"][start:end]),
            "Q promedio": np.nanmean(history["q_eval"][start:end]),
            "target promedio": np.nanmean(history["target"][start:end]),
        })
    return pd.DataFrame(rows).set_index("episodios").round(2)

pd.concat({"DQN": summarize(dqn_history), "Double DQN": summarize(ddqn_history)}, axis=1)

```

	DQN			Double DQN		
	recompensa	Q promedio	target promedio	recompensa	Q promedio	target promedio
episodios						
1-100	-60.59	-5.78	6.81	-80.23	1.22	6.70
101-200	161.04	21.69	12.01	77.77	53.88	9.98
201-300	224.53	36.53	14.28	187.82	72.40	29.66
301-400	228.23	60.84	18.98	209.65	76.15	24.88
401-500	249.34	68.81	21.38	233.54	60.14	23.92
501-600	233.78	77.01	30.33	171.63	56.76	31.35

Task 2.2

Con base en los resultados de la implementación, respondan:

- Grafiquen la recompensa promedio por episodio de DQN y Double DQN en la misma figura con media móvil de 20 episodios. ¿Cuál converge más rápido? ¿Cuál alcanza mayor recompensa final? ¿El resultado coincide con lo esperado teóricamente?
- Grafiquen el valor Q promedio estimado durante el entrenamiento para ambos algoritmos. El valor Q de DQN debería ser sistemáticamente mayor que el de Double DQN sobre los mismos estados. ¿Observan ese patrón? Si no lo observan, argumenten por qué podría no ser visible en este entorno específico.
- Implementen ahora Prioritized Experience Replay sobre Double DQN, usando $\alpha = 0.6$ y β creciente de 0.4 a 1.0. Comparen la velocidad de convergencia de Double DQN con muestreo uniforme versus Double DQN con PER. ¿En qué fase del entrenamiento es más visible la diferencia?
- Argumenten si PER sería más o menos beneficioso en el dominio real de detección de intrusiones comparado con LunarLander, considerando la rareza de los eventos críticos que analizaron en la Task 1.1c. Apoyen su argumento con evidencia de sus experimentos.

```

import matplotlib.pyplot as plt

# un color fijo por algoritmo en todas las figuras
COLORS = {"DQN": "#2a78d6", "Double DQN": "#eb6834", "Double DQN + PER": "#1baf7a"}
EPISODES = np.arange(1, NUM_EPISODES + 1)
SOLVED_THRESHOLD = 200

def moving_average(values, window=20):
    return pd.Series(values).rolling(window, min_periods=1).mean().to_numpy()

def first_episode_solved(history, window=20):
    # primer episodio en que la media móvil completa de 20 episodios llega al umbral
    ma = pd.Series(history["reward"]).rolling(window).mean().to_numpy()
    hits = np.flatnonzero(ma >= SOLVED_THRESHOLD)
    return int(hits[0]) + 1 if hits.size else None

def plot_curves(ax, histories, key, title, ylabel):
    for name, history in histories.items():
        ax.plot(EPISODES, moving_average(history[key]), color=COLORS[name], linewidth=2, label=name)
    ax.set_title(title)
    ax.set_xlabel("Episodio")
    ax.set_ylabel(ylabel)
    ax.grid(alpha=0.3)

histories = {"DQN": dqn_history, "Double DQN": ddqn_history}
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
plot_curves(axes[0], histories, "reward", "Recompensa por episodio (media móvil de 20)", "Recompensa")
axes[0].axhline(SOLVED_THRESHOLD, color="gray", linestyle="--", linewidth=1, label="Umbral 200")
plot_curves(axes[1], histories, "q_eval", "Q promedio en los 100 estados de evaluación (media móvil de 20)", "Q")
for ax in axes:
    ax.legend()
plt.tight_layout()
plt.show()

for name, history in histories.items():
    print(f"{name}: media móvil de 20 >= {SOLVED_THRESHOLD} por primera vez en el episodio {first_episode_solve")

```

```

PER_ALPHA = 0.6
PER_BETA_START, PER_BETA_END = 0.4, 1.0

class PrioritizedReplayBuffer(ReplayBuffer):
    def __init__(self, capacity, obs_dim, alpha=PER_ALPHA):
        super().__init__(capacity, obs_dim)
        self.alpha = alpha
        self.priorities = np.zeros(capacity, dtype=np.float32)
        self.max_priority = 1.0

    def add(self, state, action, reward, next_state, done):
        super().add(state, action, reward, next_state, done)
        # una transición nueva entra con la prioridad máxima para que se muestree al menos una vez
        self.priorities[(self.pos - 1) % self.capacity] = self.max_priority

    def sample(self, batch_size, beta):
        probabilities = self.priorities[:self.size].astype(np.float64) ** self.alpha
        probabilities /= probabilities.sum()
        indices = np.random.choice(self.size, batch_size, p=probabilities)
        # pesos de importancia normalizados por el máximo del minibatch
        weights = (self.size * probabilities[indices]) ** (-beta)
        weights /= weights.max()
        batch = (
            torch.as_tensor(self.states[indices], device=DEVICE),
            torch.as_tensor(self.actions[indices], device=DEVICE),
            torch.as_tensor(self.rewards[indices], device=DEVICE),
            torch.as_tensor(self.next_states[indices], device=DEVICE),
            torch.as_tensor(self.dones[indices], device=DEVICE),
        )
        return (*batch, indices, torch.as_tensor(weights, dtype=torch.float32, device=DEVICE))

    def update_priorities(self, indices, priorities):
        priorities = np.asarray(priorities, dtype=np.float32)
        self.priorities[indices] = priorities
        self.max_priority = max(self.max_priority, float(priorities.max()))

def train_double_dqn_per(eval_states, num_episodes=NUM_EPISODES, seed=SEED, alpha=PER_ALPHA):
    set_global_seed(seed)
    env = gym.make(ENV_ID)
    env.action_space.seed(seed)
    obs_dim = env.observation_space.shape[0]
    n_actions = env.action_space.n
    online_net = QNetwork(obs_dim, n_actions).to(DEVICE)
    target_net = QNetwork(obs_dim, n_actions).to(DEVICE)
    target_net.load_state_dict(online_net.state_dict())
    optimizer = torch.optim.Adam(online_net.parameters(), lr=LR)
    buffer = PrioritizedReplayBuffer(BUFFER_CAPACITY, obs_dim, alpha=alpha)
    history = {"reward": [], "q_eval": [], "target": []}
    global_step = 0
    obs, _ = env.reset(seed=seed)

    for episode in range(num_episodes):
        if episode > 0:
            obs, _ = env.reset()
            # beta crece linealmente de 0.4 a 1.0 a lo largo de los 600 episodios
            beta = PER_BETA_START + (PER_BETA_END - PER_BETA_START) * episode / (num_episodes - 1)
            episode_reward, episode_targets = 0.0, []
            done = False
            while not done:
                if random.random() < epsilon_at(global_step):
                    action = env.action_space.sample()
                else:

```

```

        with torch.no_grad():
            state_t = torch.as_tensor(obs, dtype=torch.float32, device=DEVICE)
            action = int(online_net(state_t).argmax().item())
        next_obs, reward, terminated, truncated, _ = env.step(action)
        buffer.add(obs, action, reward, next_obs, float(terminated))
        obs = next_obs
        episode_reward += reward
        done = terminated or truncated
        global_step += 1

    if buffer.size >= LEARNING_STARTS:
        states, actions, rewards, next_states, dones, indices, weights = buffer.sample(BATCH_SIZE, beta)
        targets = compute_targets(online_net, target_net, rewards, next_states, dones, double=True)
        q_taken = online_net(states).gather(1, actions.unsqueeze(1)).squeeze(1)
        td_errors = targets - q_taken
        loss = (weights * nn.functional.smooth_l1_loss(q_taken, targets, reduction="none")).mean()
        optimizer.zero_grad()
        loss.backward()
        nn.utils.clip_grad_norm_(online_net.parameters(), 10.0)
        optimizer.step()
        buffer.update_priorities(indices, td_errors.detach().abs().cpu().numpy() + 1e-6)
        episode_targets.append(targets.mean().item())

    if global_step % TARGET_UPDATE_STEPS == 0:
        target_net.load_state_dict(online_net.state_dict())

    with torch.no_grad():
        q_eval = online_net(eval_states).max(dim=1).values.mean().item()
    history["reward"].append(episode_reward)
    history["q_eval"].append(q_eval)
    history["target"].append(float(np.mean(episode_targets)) if episode_targets else np.nan)

    if (episode + 1) % 50 == 0:
        print(
            f"Double DQN + PER episodio {episode + 1}/{num_episodes} pasos {global_step} "
            f"recompensa (50) {np.mean(history['reward'][-50:]):7.1f} "
            f"Q {q_eval:6.2f} target {history['target'][-1]:6.2f} beta {beta:.2f}"
        )

    env.close()
    return online_net, {k: np.array(v) for k, v in history.items()}

```

```

ddqn_per_net, ddqn_per_history = train_double_dqn_per(eval_states)

per_histories = {"Double DQN": ddqn_history, "Double DQN + PER": ddqn_per_history}
fig, ax = plt.subplots(figsize=(8, 5))
plot_curves(ax, per_histories, "reward", "Double DQN: muestreo uniforme vs PER (media móvil de 20)", "Recompensa")
ax.axhline(SOLVED_THRESHOLD, color="gray", linestyle="--", linewidth=1, label="Umbral 200")
ax.legend()
plt.tight_layout()
plt.show()

for name, history in per_histories.items():
    print(f"{name}: media móvil de 20 >= {SOLVED_THRESHOLD} por primera vez en el episodio {first_episode_solved}")

pd.concat([name: summarize(history) for name, history in per_histories.items()], axis=1)

```

Respuestas Task 2.2

El análisis usa dos ejecuciones del mismo código con semilla 42, la **ejecución 1** es la salida guardada en el commit `e8b7278` (Task 2.1) y la **ejecución 2** es la salida actual de la celda de entrenamiento. Las celdas de código no cambiaron entre ambas, pero `set_global_seed` no fuerza algoritmos deterministas en PyTorch, así que las corridas en Colab no son reproducibles: las trayectorias ya difieren en los primeros 50 episodios. Las gráficas usan los historiales en memoria, que corresponden a la ejecución 2.

a. Recompensa promedio por bloques de 100 episodios:

Episodios	DQN (ej. 1)	Double DQN (ej. 1)	DQN (ej. 2)	Double DQN (ej. 2)
1-100	-80.19	-94.75	-60.59	-80.23
101-200	117.42	118.79	161.04	77.77
201-300	214.64	224.27	224.53	187.82
301-400	239.23	246.70	228.23	209.65
401-500	96.85	257.05	249.34	233.54
501-600	145.17	246.98	233.78	171.63

Como criterio de convergencia se toma la primera ventana de 50 episodios con recompensa promedio de al menos 200, el umbral con el que LunarLander se considera resuelto.

- **Ejecución 2 (la mostrada en el notebook):** DQN converge primero, en el episodio 250 (148,639 pasos), y Double DQN en el 350 (197,295 pasos). DQN también termina más alto: 233.78 frente a 171.63 en los últimos 100 episodios. Double DQN llegó a 256.9 en los episodios 451-500 y cayó a 110.9 en los últimos 50.
- **Ejecución 1:** ambos llegan a 200 en el episodio 200, Double DQN con menos pasos (115,934 frente a 132,008). Double DQN termina muy por encima, con 246.98 frente a 145.17. Aquí el que colapsa es DQN, que baja de 245.6 (episodios 351-400) a 28.8 (episodios 451-500).

En la ejecución 2, DQN converge más rápido y alcanza mayor recompensa final. En la ejecución 1 empatan en velocidad y Double DQN termina más alto. La comparación no es concluyente, porque la recompensa final de un mismo algoritmo cambió 88.6 puntos (DQN) y 75.4 puntos (Double DQN) entre ejecuciones, del mismo orden que la brecha entre algoritmos dentro de cada ejecución (101.8 y 62.2). Lo único que se repite en ambas es que la mejor ventana de 50 episodios de Double DQN fue algo mayor (269.3 frente a 245.6 y 256.9 frente a 251.1).

El resultado coincide solo en parte con la teoría. Double DQN corrige el sesgo del operador `max`, pero no acelera el aprendizaje por sí mismo. Lo esperado es una recompensa final igual o mayor y un entrenamiento más estable, con una ventaja pequeña en LunarLander, donde la sobreestimación es

moderada (ver b). La ejecución 1 cumple esa expectativa y la ejecución 2 no. Para afirmar cuál algoritmo es mejor harían falta al menos 5 semillas por algoritmo, comparando media y desviación estándar.

b. Valor Q promedio por bloques de 100 episodios, sobre los mismos 100 estados de evaluación:

Episodios	DQN (ej. 1)	Double DQN (ej. 1)	DQN (ej. 2)	Double DQN (ej. 2)
1-100	-8.12	2.13	-5.78	1.22
101-200	20.68	56.09	21.69	53.88
201-300	63.13	67.30	36.53	72.40
301-400	82.05	76.28	60.84	76.15
401-500	91.45	84.31	68.81	60.14
501-600	103.38	80.32	77.01	56.76

El patrón aparece solo en la segunda mitad del entrenamiento. En los episodios 101-300, Double DQN tiene el Q más alto en ambas ejecuciones. DQN lo supera desde el episodio 301 en la ejecución 1 y desde el 401 en la ejecución 2, y la brecha crece hasta 23.1 y 20.3 en los últimos 100 episodios. En esa fase también aparece la señal típica de sobreestimación: en ambas ejecuciones el Q de DQN sube en cada bloque desde el episodio 201 sin que la política mejore al mismo ritmo. En la ejecución 1 pasa de 82.05 a 103.38 mientras la recompensa cae de 239.23 a 145.17. El Q de Double DQN, en cambio, se estabiliza o baja (84.31 a 80.32 en la ejecución 1 y 76.15 a 56.76 en la ejecución 2).

Hay cinco razones por las que el patrón no es sistemático en este entorno:

1. **El Q promedio no mide el sesgo.** Cada red estima el valor de su propia política greedy, y como las políticas son distintas, una diferencia en Q puede venir de la política y no del sesgo. En los episodios 51-100 los cuatro agentes aprendieron primero a flotar hasta el límite de 1000 pasos (entre 888 y 994 pasos por episodio), y el Q de DQN se volvió negativo (-28.34 y -30.90 en el episodio 100). La diferencia temprana refleja la fase de aprendizaje de cada agente. Además, los estados de evaluación vienen de una política aleatoria y no son los que visitan los agentes entrenados. Medir la sobreestimación requiere comparar Q con el retorno descontado real desde esos estados, como hacen van Hasselt et al. (2016).
2. **Pocas acciones.** El sesgo del max crece con el número de acciones. Con errores independientes $\varepsilon_a \sim \mathcal{N}(0, \sigma^2)$, $\mathbb{E}[\max_a \varepsilon_a] \approx 1.03\sigma$ con 4 acciones, frente a $\approx 1.82\sigma$ con las 18 acciones de Atari, donde se documentó el problema.
3. **Poco ruido en los targets.** La dinámica de LunarLander es determinista (sin viento, la aleatoriedad está solo en el terreno y el empuje inicial de cada reset) y la recompensa es densa: cada paso paga el cambio en distancia, velocidad, ángulo y contacto de las patas. Con targets poco ruidosos, el max tiene menos error positivo que seleccionar.
4. **La implementación ya frena el sesgo.** La target network actualizada cada 500 pasos, la pérdida Huber y el recorte de gradiente en norma 10 limitan cómo se acumula el error en DQN y achican la

diferencia con Double DQN.

5. **Una sola semilla.** El Q de DQN en los episodios 501-600 cambió 26.4 entre ejecuciones (103.38 frente a 77.01), más que la brecha entre algoritmos dentro de cada ejecución (23.1 y 20.3). Con una semilla no se puede separar un sesgo sistemático del ruido entre corridas.

c. Las celdas de arriba implementan PER sobre Double DQN, con el mismo target, hiperparámetros, semilla y estados de evaluación que Double DQN con muestreo uniforme:

- Probabilidad de muestreo $P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}$ con $p_i = |\delta_i| + 10^{-6}$ y $\alpha = 0.6$. Cada transición nueva entra con la prioridad máxima vista, para que se muestree al menos una vez.
- Pesos de importancia $w_i = \frac{(N \cdot P(i))^{-\beta}}{\max_j w_j}$, que multiplican la pérdida Huber de cada transición. β crece linealmente de 0.4 en el primer episodio a 1.0 en el último, así que la corrección del sesgo es completa al final, cuando importa para la convergencia.
- Después de cada actualización, las prioridades del minibatch se reemplazan por el nuevo $|\delta_i|$.

Resultado experimental: pendiente. La celda de PER todavía no se ha ejecutado, así que aún no hay datos para comparar la velocidad de convergencia. [PENDIENTE: completar con el episodio en que cada variante alcanza media móvil de 200, la tabla por bloques y la fase en que se separan las curvas.]

Hipótesis a contrastar: la diferencia debería verse sobre todo en la fase temprana e intermedia (episodios 50 a 200), cuando los agentes pasan de flotar a aterrizar. En esa fase hay pocos aterrizajes exitosos (+100) en el buffer y sus errores TD son grandes, así que PER los repite mucho más que el muestreo uniforme. Al final las prioridades se vuelven más parejas y $\beta \rightarrow 1$ anula el sesgo, así que las curvas deberían juntarse. Por lo visto en (a), una sola ejecución de cada variante no basta para una conclusión firme.

d. PER sería más beneficioso en detección de intrusiones que en LunarLander. La razón no es solo que los eventos críticos sean raros, sino que fuera de ellos la recompensa casi no enseña nada.

1. **En LunarLander los eventos terminales también son raros, pero no hacen falta para aprender.** En la ejecución 2 hubo 600 episodios en 288,311 pasos (DQN) y 278,747 (Double DQN), así que como máximo el 0.21% de las transiciones terminan en aterrizaje (+100) o choque (-100). En los episodios 51-100, cuando los agentes flotaban hasta el límite de tiempo, la proporción bajó a cerca de 0.10%, el mismo orden que el 0.1% de intrusiones de la Task 1.1c. Aun así, ambos algoritmos superaron 200 puntos con muestreo uniforme, porque LunarLander paga en cada paso el acercamiento a la plataforma y casi todas las transiciones llevan señal útil, en detección de intrusiones, ignorar tráfico normal (al menos 99.9% de los pasos) da recompensa 0, y lo que distingue una buena política de una mala (-200 por intrusión ignorada, +40 y +80 por detectarla y contenerla, Task 1.1d) está casi todo en el 0.1% restante.
2. **En LunarLander la frecuencia la controla el agente; en intrusiones no.** Al mejorar la política, los episodios se acortaron (DQN, ejecución 2: 963 pasos por episodio en los episodios 51-100 y 306 en

los 551-600), así que los eventos terminales pasaron de cerca de 0.10% a 0.33% de las transiciones. La tasa de intrusiones la fijan los atacantes y sigue en 0.1% o menos por bueno que sea el agente, así que el desbalance dura todo el entrenamiento.

3. **Cuánto cambia el muestreo.** Con el minibatch de 64 de la implementación y $p = 0.001$, el muestreo uniforme incluye en promedio $64(0.001) = 0.064$ intrusiones por minibatch, y $0.999^{64} \approx 93.8\%$ de los minibatches no incluye ninguna. Con PER, una transición cuyo error TD es k veces el típico se muestrea $k^{0.6}$ veces más: 3.98 veces si $k = 10$ y 15.8 veces si $k = 100$. Si una intrusión ignorada (-200) produce errores TD entre 10 y 100 veces mayores que el tráfico normal (costos de 0 a 15), su participación en los minibatches sube de 0.1% a entre 0.4% y 1.6%. Los pesos de importancia con $\beta \rightarrow 1$ corrigen ese sobremuestreo en la actualización, para que el agente no aprenda a bloquear de más solo por haber visto más intrusiones, lo que empeoraría el 8% de falsos positivos.
4. **El muestreo uniforme fue inestable incluso en LunarLander.** En cada ejecución uno de los algoritmos colapsó después de converger: DQN en la ejecución 1 (de 245.6 a 28.8, con episodios de 214 pasos) y Double DQN en la ejecución 2 (de 256.9 a 110.9, con episodios de 173 pasos). Recompensa baja con episodios cortos es consistente con más choques, una explicación conocida es que, cuando la política ya es buena, el buffer de 50,000 transiciones (unos 160 a 180 episodios recientes) contiene casi solo aterrizajes, el muestreo uniforme casi no vuelve a mostrar estados de choque y la red los olvida. PER los volvería a muestrear en cuanto su error TD creciera. No medimos la composición del buffer, así que esto es una interpretación y no una prueba, en detección de intrusiones el efecto sería peor, porque los eventos críticos son raros desde el inicio y no solo cuando el agente ya es bueno.
5. **Evidencia directa de PER.** Si en (c) PER acelera sobre todo la fase temprana, cuando los aterrizajes exitosos todavía son raros en el buffer, eso respalda que su beneficio crece con la rareza de la señal. [PENDIENTE: citar el resultado de (c).]

Task 2.3

Investigación bibliográfica y dictamen técnico.

a. Busquen y lean un paper publicado entre 2022 y 2025 que aplique DQN, Double DQN, Dueling DQN, PER, o Rainbow a un problema de ciberseguridad, detección de anomalías, o gestión de redes. El paper debe ser de IEEE Transactions on Information Forensics and Security, NeuroIPS, ICML, USENIX Security, o similar. Escriban un resumen técnico de media página con: el problema que resuelve, qué variante de DQN usa y por qué, los resultados principales, y una reflexión sobre si las limitaciones de DQN que identificaron en la Task 1 fueron resueltas en ese trabajo.

b. Redacten un dictamen técnico de un párrafo dirigido al equipo directivo de la empresa de ciberseguridad. El dictamen debe argumentar si recomiendan proceder con DQN o alguna de sus variantes para el sistema de detección de intrusiones, qué variante específica recomiendan y por qué, y qué pasos adicionales serían necesarios antes de un despliegue en producción. El dictamen debe estar respaldado por evidencia de sus experimentos, no solo por argumentos teóricos.